INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

UNIDAD DE APRENDIZAJE

Ingeniería de Pruebas

EXAMEN

Caja Blanca

SECUENCIA Y PERIODO

6NM61 2026-2

BOLETA

2021601769

ALUMNO

Gonzalez Calzada Maximiliano

PROFESORA

González Arroyo Lilia

FECHA

10 de Marzo de 2026

1. Instrucciones

INGENIERÍA DE PRUEBAS
1ER DEPARTAMENTAL PRÁCTICO

Boleta, Nombre, secuencia, fecha

1. De acuerdo con los dos ejercicios plantea una prueba de CAJA BLANCA por caminos básicos. Sigue el proceso visto en clase:
   A. A partir del código construye EL GRAFO.
   B. Revisa que fórmula correspondiente a la COMPLEJIDAD CICLOMÁTICA utilizarás y calcúlala.
   C. Identifica el conjunto de CAMINOS BÁSICOS y créalos.
   D. Diseña los CASOS DE PRUEBA correspondientes.

2. Primer código

```
Si (edad >= 18)
   Si (tieneMembresia == true)
      Mensaje "Acceso total"
   Sino
      Si (codigoPromo == "VIP")
         Mensaje "Acceso invitado VIP"
      Sino
         Mensaje "Acceso denegado: requiere membresía"
Sino
   Mensaje "Acceso denegado: menor de edad"
Fin
```

3. Segundo Código. Se debe probar un bucle, aquí el desafío es que un bucle puede ejecutares cero, una o muchas veces, sin embargo, para cumplir con la complejidad ciclomática, solo necesitamos probar las rutas lógicas que definen la estructura. Por lo tanto, realiza la prueba de caja blanca sobre el siguiente código "suma de números pares de una lista hasta encontrar el número negativo o terminar la lista"

```
suma = 0
i = 0
mientras (i < largo_lista):  # Decisión A
   si (lista[i] < 0):      # Decisión B
      break (salir)
   si (lista[i] % 2 == 0):  # Decisión C
      suma += lista[i]
   i += 1
retornar suma
```

4.- Crea un documento llamado Caja Blanca_nombre.pdf donde agregues todos los pasos de caja blanca para cada uno de los códigos. Adjunta tu documento en la actividad programada.

2. Primer código

2.1. Código a analizar (sintaxis traducida a TypeScript)

```
if(edad >= 18){ // 1
  if(tieneMembresia == true){ // 2
    console.log("Acceso total"); // 3
  } else if (codigoPromo == "VIP"){ // 4
      console.log("Acceso invitado VIP"); // 5
    } else {
      console.log("Acceso denegado: requiere membresía"); // 6
    }
} else {
    console.log("Acceso denegado: menor de edad"); // 7
}
// 8 (Fin)
```

2.2. A. Grafo de flujo

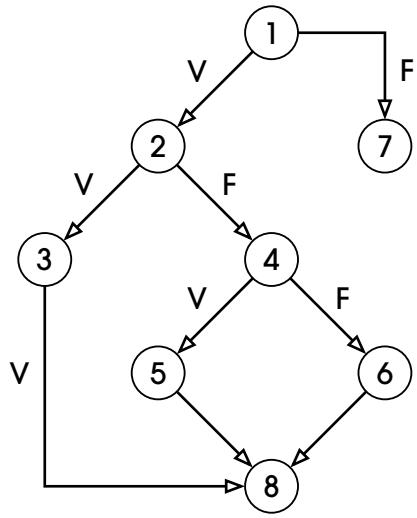

Figura 1: Grafo del primer código

2.3. B. Complejidad Ciclomática

Para calcular la complejidad ciclomática $V(G)$ podemos utilizar tres fórmulas distintas que nos darán el mismo resultado:

1. **Fórmula de Aristas y Nodos:** $V(G) = E - N + 2$
    - Aristas (E): 10
    - Nodos (N): 8

- $V(G) = 10 - 8 + 2 = 4$

2. **Fórmula de Nodos Predicado:** $V(G) = P + 1$
   - Nodos Predicado (P, nodos con condición lógica): 3 (nodos 1, 2 y 4)
   - $V(G) = 3 + 1 = 4$
3. **Fórmula de Regiones:** $V(G) = R$
   - Regiones cerradas en el grafo: 3
   - Región abierta (exterior): 1
   - $V(G) = 3 + 1 = 4$

Resultado: La complejidad ciclomática es de **4**, lo que significa que debemos encontrar 4 caminos básicos e independientes.

2.4. C. Caminos Básicos

1. **Camino 1:** 1 $\rightarrow$ 7 $\rightarrow$ 8
2. **Camino 2:** 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 8
3. **Camino 3:** 1 $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 8
4. **Camino 4:** 1 $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 8

2.5. D. Casos de Prueba

Camino	`edad`	`tieneMembresia`	`codigoPromo`	**Salida Esperada**	**Salida Real**
1	17	Falso	«–»	«Acceso denegado: menor de edad»	«Acceso denegado: menor de edad»
2	20	Verdadero	«–»	«Acceso total»	«Acceso total»
3	25	Falso	«VIP»	«Acceso invitado VIP»	«Acceso invitado VIP»
4	30	Falso	«REGULAR»	«Acceso denegado: requiere membresía»	«Acceso denegado: requiere membresía»

3. Segundo código

3.1. Código a analizar (sintaxis traducida a TypeScript)

```
let suma = 0; // 1
let i = 0; // 2
while (i < lista.length) { // 3
    if (lista[i] < 0) { // 4
        break; // 5
    }
    if (lista[i] % 2 === 0) { // 6
        suma += lista[i]; // 7
    }
    i++; // 8
}
return suma; // 9
```

3.2. A. Grafo de flujo

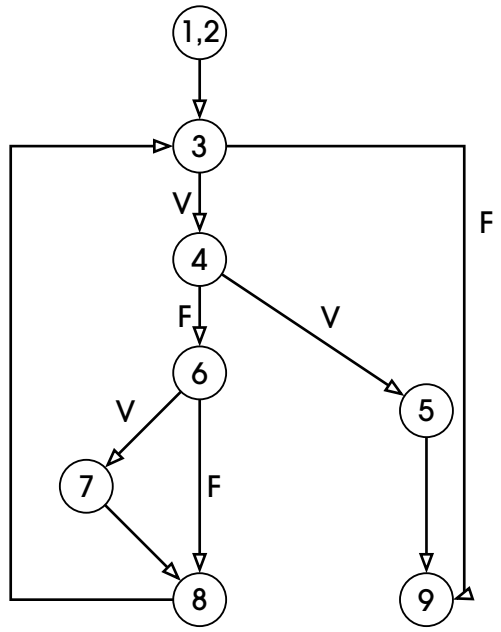

Figura 2: Grafo del segundo código

3.3. B. Complejidad Ciclomática

Para calcular la complejidad ciclomática $V(G)$:

1. **Fórmula de Aristas y Nodos:** $V(G) = E - N + 2$
    - Aristas (E): 10

- Nodos (N): 8
- $V(G) = 10 - 8 + 2 = 4$

2. **Fórmula de Nodos Predicado:** $V(G) = P + 1$
   - Nodos Predicado (P): 3 (nodos 3, 4 y 6)
   - $V(G) = 3 + 1 = 4$

Resultado: La complejidad ciclomática es de **4**, resultando en 4 caminos básicos.

3.4. C. Caminos Básicos

Considerando los nodos de nuestro grafo:

1. **Camino 1:** (1,2) $\rightarrow$ 3 $\rightarrow$ 9
2. **Camino 2:** (1,2) $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 9
3. **Camino 3:** (1,2) $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$ 9
4. **Camino 4:** (1,2) $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$ 9

3.5. D. Casos de Prueba

Camino	`lista`	Condición que cumple	Salida Esperada	Salida Real
1	`[]`	Lista vacía (falla `i < lista.length` inicial)	`0`	`0`
2	`[-1]`	Encuentra número negativo, hace `break`	`0`	`0`
3	`[1]`	Itera número impar (no lo suma), termina	`0`	`0`
4	`[2]`	Itera número par (lo suma), termina	`2`	`2`

4. Conclusión

Se aplicó la técnica de prueba de Caja Blanca para ambos bloques de código, comenzando por trazar su grafo de flujo para visualizar los caminos de ejecución. Posteriormente, se calculó la complejidad ciclomática logrando identificar la cantidad exacta de caminos básicos independientes (4 en ambos casos). Finalmente, se diseñaron los casos de prueba que garantizan la cobertura de sentencias y ramas al recorrer al menos una vez todas las aristas y nodos del grafo.

5. Código usado para construir este documento

```
// Importamos la librería 'cetz' (Canvas para Typst) que nos permite dibujar formas y grafos.
#import "@preview/cetz:0.4.2"
#import "portada-template.typ": portada

// Definimos una variable que contiene un arreglo con los nombres de los integrantes del equipo.
#let integrantes = (
  "Gonzalez Calzada Maximiliano"
)

// Llamamos a la función 'portada' pasándole todos los datos requeridos (clase, boleta, nombres, etc.) para generar la página inicial.
#portada(
  "UNIDAD DE APRENDIZAJE",
  "EXAMEN",
  "SECUENCIA Y PERIODO",
  "BOLETA",
  "ALUMNO",
  "PROFESORA",
  "FECHA",
  "Ingeniería de Pruebas",
  "Caja Blanca",
  "6NM61 2026-2",
  "2021601769",
  integrantes,
  "González Arroyo Lilia",
  "10 de Marzo de 2026",
)

// Las funciones '#set' cambian la configuración global de un elemento.
// Aquí establecemos la fuente principal, el idioma (es) y el grosor del texto para todo el documento.
```

```
#set text(
  font: "ITC Avant Garde Gothic",
  lang: "es",
  weight: "semibold",
)

// Configuramos el tamaño de la página (tamaño carta), definimos los márgenes de 3cm y
2.5cm acordes a formato estándar, y activamos el número de página.
#set page(
  paper: "us-letter",
  margin: (left: 3cm, top: 2.5cm, right: 2.5cm, bottom: 2.5cm),
  numbering: "1",
)

// Obliga a insertar un salto hacia la página siguiente.
#pagebreak()
// Ajustamos las reglas de los párrafos (par): 'justify: true' para texto justificado
y 'leading' ajusta el interlineado.
// También configuramos listas indexadas y la forma de la numeración de títulos.
#set par(justify: true, leading: 1.4em)
#set heading(numbering: "1.")
#set list(indent: 1.5em)

// Cambiamos temporalmente el tamaño de página a 'auto' para que la página se adapte
dinámicamente al tamaño horizontal y vertical del contenido (ej. nuestro pdf anexo).
#set page(
  width: auto,
  height: auto,
  margin: 1cm,
)

// Con un signo de igual '=' creamos un título o encabezado de primer nivel (equivale
a H1 en HTML HTML).
= Instrucciones
```

```
// Cargamos la información cruda (bytes) del archivo PDF externo.
#let instructions = read("media/Examen-practico_IP.pdf", encoding: none)
// Construimos la imagen del PDF en el documento leyendo esa data.
#image(instructions, width: auto, height: auto)

#set page(
  paper: "us-letter",
  margin: (left: 3cm, top: 2.5cm, right: 2.5cm, bottom: 2.5cm),
  numbering: "1",
)

= Primer código

// Con '==' creamos un título de segundo nivel (subtítulo).
== Código a analizar (sintaxis traducida a TypeScript)
// Usamos bloques de código con acentos graves especificando un lenguaje como ts
(TypeScript) para obtener colores y resaltado de la sintaxis.
if(edad >= 18){ // 1
  if(tieneMembresia == true){ // 2
    console.log("Acceso total"); // 3
  } else if (codigoPromo == "VIP"){ // 4
      console.log("Acceso invitado VIP"); // 5
    } else {
      console.log("Acceso denegado: requiere membresía"); // 6
    }
} else {
    console.log("Acceso denegado: menor de edad"); // 7
}
// 8 (Fin)
== A. Grafo de flujo

// Utilizamos el constructor especial '#figure' para envolver algo gráfico en la hoja,
lo cual nos permite ponerle una leyenda / texto ilustrativo centrado ('caption') al
pie del gráfico.
```

```
#figure(
// Iniciamos el entorno o lienzo de dibujo 'cetz'. A partir de aquí entramos en su
propio lenguaje lógico de dibujo.
  cetz.canvas({
    import cetz.draw: *

// Declaramos un estilo visual base que ocuparemos más adelante repetidas veces (el
radio del círculo, el grosor del borde, y fondo blanco).
    let node-style = (radius: 0.35, stroke: 0.7pt, fill: white)

// Dibujamos el primer círculo en la coordenada matemática (x: 0, y: 5) de nuestro
lienzo. Al ponerle nombre (name: "1") luego podremos referenciarlo desde las flechas y
textos.
    // Con '..node-style' desempacamos/heredamos la configuración visual que
escribimos arriba.
    circle((0, 5), name: "1", ..node-style)
// Escribimos el contenido textual estricto sobre lo que se haya etiquetado como "1"
hace un momento, en este caso, dibujando el [1].
    content("1", [1])

    circle((-1.5, 3.5), name: "2", ..node-style)
    content("2", [2])

    circle((1.5, 3.5), name: "7", ..node-style)
    content("7", [7])

    circle((-3, 2), name: "3", ..node-style)
    content("3", [3])

    circle((0, 2), name: "4", ..node-style)
    content("4", [4])

    circle((-1.5, 0.5), name: "5", ..node-style)
    content("5", [5])
```

```
    circle((1.5, 0.5), name: "6", ..node-style)
    content("6", [6])

    circle((0, -1), name: "8", ..node-style)
    content("8", [8])

// Renderizamos una línea para conectar los elementos con nombres "1" y "2". Con mark
= (end: ">") Typst dibujará una terminación de flecha geométrica al extremo que entra.
    line("1", "2", mark: (end: ">"), name: "e12")
    content("e12", [V], anchor: "south-east", padding: 0.1)

    line("1", "7", mark: (end: ">"), stroke: 0pt)

    line("2", "3", mark: (end: ">"), name: "e23")
    content("e23", [V], anchor: "south-east", padding: 0.1)

    line("2", "4", mark: (end: ">"), name: "e24")
    content("e24", [F], anchor: "south-west", padding: 0.1)

    line("4", "5", mark: (end: ">"), name: "e45")
    content("e45", [V], anchor: "south-east", padding: 0.1)

    line("4", "6", mark: (end: ">"), name: "e46")
    content("e46", [F], anchor: "south-west", padding: 0.1)

// Éste trazo de varios argumentos funciona en segmentos quebrados (ortogonales):
parte desde la coordenada inferior del nodo ("3.south"), viaja hacia el punto de
inflexión vacío (-3, -1), y finalmente busca entrar al oeste ("8.west") del último
nodo.
    line("3.south", (-3, -1), "8.west", mark: (end: ">"))
    content((-3.1, 0.5), [V], anchor: "east", padding: 0.1)

    line("5", "8", mark: (end: ">"))
```

```
    line("6", "8", mark: (end: ">"))

    line("1.east", (1.5, 5), "7.north", mark: (end: ">"))
    content((1.6, 4.5), [F], anchor: "west", padding: 0.1)
  }),
  caption: [Grafo del primer código],
)<primer-grafo>
```

== B. Complejidad Ciclomática

// Typst es genial con notación matemática simple. Podemos rodear expresiones con símbolos de dólar ('$') para renderizar fórmulas visuales bien escritas (ej. $V(G)$).

Para calcular la complejidad ciclomática $V(G)$ podemos utilizar tres fórmulas distintas que nos darán el mismo resultado:

1. *Fórmula de Aristas y Nodos:* $V(G) = E - N + 2$
   - Aristas (E): 10
   - Nodos (N): 8
   - $V(G) = 10 - 8 + 2 = 4$

2. *Fórmula de Nodos Predicado:* $V(G) = P + 1$
   - Nodos Predicado (P, nodos con condición lógica): 3 (nodos 1, 2 y 4)
   - $V(G) = 3 + 1 = 4$

3. *Fórmula de Regiones:* $V(G) = R$
   - Regiones cerradas en el grafo: 3
   - Región abierta (exterior): 1
   - $V(G) = 3 + 1 = 4$

Resultado: La complejidad ciclomática es de *4*, lo que significa que debemos encontrar 4 caminos básicos e independientes.

== C. Caminos Básicos

1. *Camino 1:* 1 $->$ 7 $->$ 8
2. *Camino 2:* 1 $->$ 2 $->$ 3 $->$ 8
3. *Camino 3:* 1 $->$ 2 $->$ 4 $->$ 5 $->$ 8
4. *Camino 4:* 1 $->$ 2 $->$ 4 $->$ 6 $->$ 8

== D. Casos de Prueba

```
// El componente '#table' agrupa las cosas en celdas matriciales. Definiendo las columnas como vector (auto, auto...) estipularemos cuántas colécciones de celdas queremos y que se auto-ajusten su ancho.
// Luego, por simple orden de tabulado vertical pasamos las celdas una a una como [Valor] por corchetes.
#table(
  columns: (auto, auto, auto, auto, auto, auto),
  align: center,
  [*Camino*], [*`edad`*], [*`tieneMembresia`*], [*`codigoPromo`*], [*Salida Esperada*], [*Salida Real*],
  [1], [17], [Falso], ["-"], ["Acceso denegado: menor de edad"], ["Acceso denegado: menor de edad"],
  [2], [20], [Verdadero], ["-"], ["Acceso total"], ["Acceso total"],
  [3], [25], [Falso], ["VIP"], ["Acceso invitado VIP"], ["Acceso invitado VIP"],
  [4], [30], [Falso], ["REGULAR"], ["Acceso denegado: requiere membresía"], ["Acceso denegado: requiere membresía"],
)

#pagebreak()
```

= Segundo código

== Código a analizar (sintaxis traducida a TypeScript)

```
let suma = 0; // 1
let i = 0; // 2
while (i < lista.length) { // 3
    if (lista[i] < 0) { // 4
```

```
        break; // 5
    }
    if (lista[i] % 2 === 0) { // 6
        suma += lista[i]; // 7
    }
    i++; // 8
}
return suma; // 9
```

== A. Grafo de flujo

```
#figure(
  cetz.canvas({
    import cetz.draw: *

    let node-style = (radius: 0.35, stroke: 0.7pt, fill: white)

    circle((0, 6), name: "1", ..node-style)
    content("1", [1,2])

    circle((0, 4.5), name: "3", ..node-style)
    content("3", [3])

    circle((0, 3), name: "4", ..node-style)
    content("4", [4])

    circle((3, 1), name: "5", ..node-style)
    content("5", [5])

    circle((0, 1.5), name: "6", ..node-style)
    content("6", [6])

    circle((-1.5, 0), name: "7", ..node-style)
    content("7", [7])
```

```
circle((0, -1.5), name: "8", ..node-style)
content("8", [8])

circle((3, -1.5), name: "9", ..node-style)
content("9", [9])

// Edges
line("1", "3", mark: (end: ">"))

line("3", "4", mark: (end: ">"), name: "e34")
content("e34", [V], anchor: "east", padding: 0.1)

line("3.east", (3.5, 4.5), (3.5, -1.5), "9.east", mark: (end: ">"))
content((3.6, 3.5), [F], anchor: "west", padding: 0.1)

line("4", "5", mark: (end: ">"), name: "e45")
content("e45", [V], anchor: "south-west", padding: 0.1)

line("4", "6", mark: (end: ">"), name: "e46")
content("e46", [F], anchor: "east", padding: 0.1)

line("5", "9", mark: (end: ">"))

line("6", "7", mark: (end: ">"), name: "e67")
content("e67", [V], anchor: "south-east", padding: 0.1)

line("6", "8", mark: (end: ">"), name: "e68")
content("e68", [F], anchor: "west", padding: 0.1)

line("7", "8", mark: (end: ">"))

// Bucle (Loop back)
line("8.west", (-2.5, -1.5), (-2.5, 4.5), "3.west", mark: (end: ">"))
```

```
  }),
  caption: [Grafo del segundo código],
)<segundo-grafo>

== B. Complejidad Ciclomática

Para calcular la complejidad ciclomática $V(G)$:

1. *Fórmula de Aristas y Nodos:* $V(G) = E - N + 2$
   - Aristas ($E$): 10
   - Nodos ($N$): 8
   - $V(G) = 10 - 8 + 2 = 4$

2. *Fórmula de Nodos Predicado:* $V(G) = P + 1$
   - Nodos Predicado ($P$): 3 (nodos 3, 4 y 6)
   - $V(G) = 3 + 1 = 4$

*Resultado:* La complejidad ciclomática es de *4*, resultando en 4 caminos básicos.

== C. Caminos Básicos

Considerando los nodos de nuestro grafo:
1. *Camino 1:* (1,2) $->$ 3 $->$ 9
2. *Camino 2:* (1,2) $->$ 3 $->$ 4 $->$ 5 $->$ 9
3. *Camino 3:* (1,2) $->$ 3 $->$ 4 $->$ 6 $->$ 8 $->$ 3 $->$ 9
4. *Camino 4:* (1,2) $->$ 3 $->$ 4 $->$ 6 $->$ 7 $->$ 8 $->$ 3 $->$ 9

== D. Casos de Prueba

#table(
  columns: (auto, auto, auto, auto, auto),
  align: center,
  [*Camino*], [*`lista`*], [*Condición que cumple*], [*Salida Esperada*], [*Salida
Real*],
```

```
  [1], [`[]`], [Lista vacía (falla `i < lista.length` inicial)], [`0`], [`0`],
  [2], [`[-1]`], [Encuentra número negativo, hace `break`], [`0`], [`0`],
  [3], [`[1]`], [Itera número impar (no lo suma), termina], [`0`], [`0`],
  [4], [`[2]`], [Itera número par (lo suma), termina], [`2`], [`2`],
)
```

= Conclusión

Se aplicó la técnica de prueba de Caja Blanca para ambos bloques de código, comenzando por trazar su grafo de flujo para visualizar los caminos de ejecución. Posteriormente, se calculó la complejidad ciclomática logrando identificar la cantidad exacta de caminos básicos independientes (4 en ambos casos). Finalmente, se diseñaron los casos de prueba que garantizan la cobertura de sentencias y ramas al recorrer al menos una vez todas las aristas y nodos del grafo.

```
#pagebreak()
```

= Código usado para construir este documento (easter egg! [no voy a caer en la recursión])